郑州大学管理学院
本科毕业论文

题目：基于 BERT 模型的在线评论效价甄别研究

学生姓名及学号　金铭 202032060611

专　　业　市场营销

班　　级　2020 级 2 班

指导教师及职称　丁海欣 讲师

2024 年 5 月 26 日

目录

摘　　要 1

Abstract 2

1 绪　论 3

1.1 研究背景 3

1.1.1 实践背景 3

1.1.2 理论背景 4

1.2 研究意义 4

1.2.1 理论意义 4

1.2.2 现实意义 4

1.3 研究内容和框架 5

2 理论基础 7

2.1 在线评论分析的国内外研究现状 7

2.2 文本情感分类相关研究综述 7

2.3 BERT 模型的机制与原理介绍 8

2.3.1 双向 Transformer Encoder 8

2.3.2 多头注意力机制 8

2.3.3 嵌入（Embedding） 9

2.3.4 掩码（Masked Language Model） 9

2.3.5 NSP（Next Sentence Prediction） 9

3 基于 BERT 的情感分析和反讽检测模型 10

3.1 模型概述 10

3.2 预训练模型 10

3.2.1 在线评论文本数据转换 10

3.2.2 编码和特征提取 11

3.3 模型微调 11

3.4 模型评估指标的选择 12

4 实验与结果分析 13

4.1 数据来源与预处理 13

4.2 实验环境 16

4.3 BERT 模型实验结果 16

4.4 对比实验结果 17

5 总结与展望 19

5.1 研究成果与结论 19

5.2 研究意义 19

5.3 不足与展望 19

致　谢 20

参考文献 **21**
附　录 **23**

摘　　要

当前，许多学者已经利用机器学习技术进行在线评论的情感分类，也有一些学者专注于反讽语义的监测，这些研究都取得了一定的成果。然而，站在现实的角度来看，许多在线评论并不仅仅是简单的积极或消极情感表达，其中很多评论可能暗含讽刺语义。讽刺既可以表达消极情绪，也可以表达积极情绪，其多样性使得单一的情感分类模型往往无法准确捕捉到评论的真实含义。因此，构建一个能够同时检测在线评论情感和反讽的模型具有重要意义。通过综合考虑情感和反讽语义，可以更全面地理解和分析在线评论，为用户提供更准确、更深入的信息。然而，目前很少有学者在这个方向进行深入研究，因此本研究的重要性在于填补了这一领域的空白，为在线评论情感和反讽分析提供了新的方法和视角。

本文旨在利用 BERT 模型对游戏评论进行情感分类和反讽检测。首先，使用自然语言处理（NLP）技术对文本进行预处理和标注，然后将数据输入到 BERT 预训练模型中进行微调。基于 GitHub 上的 BERT 预训练模型，分别构建了情感分类和反讽检测模型。对 Steam 游戏评论数据集进行了训练和测试，并分析其性能。实验结果显示，模型在情感分类和反讽检测任务中均取得了令人满意的性能，能够有效识别评论的情感倾向和反讽语义。随后，将该模型与不同的机器学习模型进行了对比实验，以验证其优越性。这一研究为利用自然语言处理技术对游戏在线评论进行情感和反讽分析提供了可行的方法和技术支持，不仅提升了分析的效率和准确性，还为商家和营销提供了有价值的方法，以更好地理解用户反馈和改进产品策略。

关键词：机器学习；反讽检测；情感分类；游戏评论

Abstract

Currently, many scholars have used machine learning technology to classify sentiments in online comments, and some scholars have focused on monitoring irony semantics. These studies have achieved certain results. However, from a realistic perspective, many online comments are not simply expressions of positive or negative emotions, and many of them may contain sarcastic semantics. Sarcasm can express both negative and positive emotions, and its diversity makes a single emotion classification model often unable to accurately capture the true meaning of a comment. Therefore, it is of great significance to build a model that can simultaneously detect emotion and irony in online reviews. By taking both emotion and irony semantics into account, online reviews can be more comprehensively understood and analyzed, providing users with more accurate and in-depth information. However, currently few scholars have conducted in-depth research in this direction, so the importance of this study lies in filling the gap in this field and providing new methods and perspectives for the analysis of emotion and irony in online comments.

This article aims to use the BERT model to perform sentiment classification and irony detection on game reviews. First, the text is preprocessed and annotated using natural language processing (NLP) technology, and then the data is input into the BERT pre-trained model for fine-tuning. Based on the BERT pre-training model on GitHub, emotion classification and irony detection models were constructed respectively. The Steam game review dataset is trained and tested, and its performance is analyzed. Experimental results show that the model achieves satisfactory performance in both emotion classification and irony detection tasks, and can effectively identify the emotional tendency and irony semantics of comments. Subsequently, comparative experiments were conducted between this model and different machine learning models to verify its superiority. This research provides feasible methods and technical support for using natural language processing technology to analyze emotion and irony in online game reviews. It not only improves the efficiency and accuracy of analysis, but also provides valuable methods for merchants and marketing to Better understand user feedback and improve product strategy.

Key words: machine learning；irony detection；emotion classification；game review

1 绪　论

1.1 研究背景

1.1.1 实践背景

情感分析技术作为自然语言处理的一种应用，能够迅速抓取、整合互联网上海量的非结构化评论数据，挖掘带有情感倾向的主观性文本，实现分析、处理、归纳和推理的过程[1]。近些年，文本情感分析被广泛应用于商业领域、舆情监测、信息检索和人工智能等方面。

文本情感分析在商业领域常用于分析用户的评论来获取有效的信息，从而为商业决策提供支持。通过分析社交媒体和在线评论平台的帖子，企业可以实时监控品牌声誉，并在负面情感出现时及时采取公关手段，维护品牌形象。同时，企业也可以通过在线评论了解竞争对手的品牌评价，理解市场的最新动向和消费者对竞争品牌的看法，从而调整自身的竞争策略。商家也在利用社交媒体进行广告投放和宣传商品，并通过与消费者的有效线上互动维持品牌形象。文本情感分析还可以帮助企业和组织了解公众舆论动态、预测趋势、预防危机。很多政府机构和媒体都在社交平台、论坛、博客等运营官方账号，通过在线评论分析技术实时监测公众对特定的事件和人物的情感倾向，了解公众关注的焦点和态度，从而及时控制事件的走向和影响范围。在信息检索领域，文本情感分析技术可以显著提升用户体验和检索效果。在社交媒体和网购平台，大数据可以通过分析用户的搜索和浏览历史为用户推荐符合其情感倾向的内容。文本情感分析也被广泛利用在人工智能领域。为了提高效率和降低成本，很多平台选择使用智能客服系统。智能客服系统会通过情感分析技术识别用户的不同情绪，从而做出相应的回答。一些智能软件，如 Siri、小 e、天猫精灵等系统提供了更加人性化的互动体验。2016 年由 Hanson Robotics 公司开发的 Sophia 仿真机器人，可以理解和生成语言，参与自然的对话。Sophia 不但可以通过音频识别人类的情感，还可以通过识别人类的面部表情做出相应的回应。

随着互联网的飞速发展，社交媒体和各种在线平台成为了用户线上交流的主要媒介，用户可以在自己感兴趣的领域和网友进行在线讨论，发表自己的意见和观点。然而，拥有庞大用户量的社交媒体平台也成为了谣言信息泛滥的场所，大量谣言迅速传播不仅危害网络空间安全，还威胁到现实世界的政治、经济、社会等方面的秩序[2][3]。因此，如何衡量线评论质量和情感的表达成为了亟需解决的问题。在线评论往往具有强烈的主观性，其中蕴含丰富的感情色彩[1]。例如，图 1-1 所展示的是一些游戏平台的在线评论，我们可以看出，有些评论的内容和用户的评价倾向是一致的，而有些用户却选择使用反讽的方式给出好评或差评。由此可见，单纯依靠评论的字面意思和用户的进行情感分析会导致误判。这种反讽评论在实际应用中相当普遍，尤其是在社交媒体和在线购物平台上，用户常常用反讽的方式表达不满或讽刺某些产品和服务。发现包含讽刺和反语的句子能极大提高情感分析性能，却也是自然语言处理领域面临的最艰巨的任务之一[4]。

图 1-1 在线评论实例

1.1.2 理论背景

文本情感分析是利用自然语言处理和文本挖掘技术，从非结构化文本中提取用户表达的情感信息，从而助力研究者判断发布文本用户对特定主题的情感倾向[5]。传统的文本情感分析主要利用机器学习方法，如逻辑回归、朴素贝叶斯、支持向量机（SVM）等模型。这些方法通过提取特征向量，利用词袋模型和 TF-IDF 等方法进行情感分类。传统机器学习实现简单，适用于小规模数据集，但由于依赖特征工程，有时难以捕捉上下文信息从而处理复杂的情感表达。

深度学习方法在文本情感分析中表现出色，如递归神经网络（RNN）、卷积神经网络（CNN）和长短期记忆模型（LSTM）等，可以自动提取文本特征，并善于捕捉上下文信息。因此，深度学习模型能够处理大规模的数据集，并且能够捕捉更为复杂的情感模式。然而，深度学习模型需要大量的标注数据进行训练，对计算机资源和配置要求较高，并且训练时间长，且难以解释模型内部的机制，也就是常说的“黑箱”。近年来，预训练模型如 BERT 模型等通过在大规模的语料库上进行预训练，能够学习到丰富的语言表示，并可以通过在下游任务中进行微调从而捕捉到上下文的信息。

尽管现有模型在文本情感分析方面较为成熟，但在应对多情感混合表达时仍表现出不足，在特定情景下的情感强度识别也存在困难，比如在处理讽刺和隐喻等复杂语言现象时准确性仍有限。因此，需要综合考虑不同情感表达形式的分类方法，有助于更全面地理解文本中的情感信息，并提高情感分析的准确性和普适性。

1.2 研究意义

1.2.1 理论意义

在线评论中经常包含多层次的情感信息，而传统模型和研究通常难以兼顾多任务处理。本研究基于 BERT 模型，提供了一个可以同时进行情感分类和反讽监测的情感分析方法，丰富了文本情感分析领域的研究内容。本文的研究还为处理复杂情感分析提供了经验和参考，在一定程度上拓展了文本情感分析的研究视角和研究方法。本研究所提出的多任务学习方法即提高了模型的性能，又减少了训练时间和计算资源的消耗。通过这种多任务学习方法，可以更有效地识别和分类多层次的情感信息，是对深度学习研究的扩展。

此外，本研究对社交媒体和在线评论的研究具有重要的理论意义。通过引入多任务学习方法，为分析复杂情感信息提供了新的思路和方法，有助于更全面地理解和解释社交媒体上的用户情感表达。这不仅推动了情感分析技术的发展，也为社交媒体研究和在线评论分析领域的探索提供了新的方向和参考，有助于更准确地捕捉和理解用户的情感态度和行为模式。

1.2.2 现实意义

对企业和商家而言，应用在线评论情感分析技术可以更好地捕捉消费者的真实情感和需求，从而及时改进产品或服务的缺点。同时，这也有助于商家优化客户关系，通过对用户进行个性化的服务提高用户的满意度和忠诚度。文本情感分析还可以帮助企业及时发现负面评论，在品牌出现危机时可以快速响应，从而保护品牌形象。使用文本情感分析工具还可以提高企业的运营效率，让商家更快速高效地处理海量信息，将精力和预算花在产品创新和战略决策上。

对于消费者而言，文本情感分析技术可以为他们提供更加客观准确的信息，帮助消费者进行更加理性的决策。同时，消费者可以更好地向商家传递自己对产品和服务的真实情感，从而推动产品和服务的改进，形成良性循环。当消费者意识到提出评价后可以得到良好的反馈，会增强消费者的主人翁意识，从而增强对商家和品牌的认同感和忠诚度。

1.3 研究内容和框架

本文旨在利用机器学习模型和自然语言处理技术对在线评论进行情感分析。第一部分为绪论，主要介绍在线评论的现状和一些存在的问题，并提出研究的目标。第二部分为理论基础，介绍该研究领域的国内外研究现状以及机器学习和在线评论甄别的原理和理论。第三部分为本文的研究设计，主要是评论数据的收集与处理和模型的构建。利用 Python 爬虫技术收集 Steam 平台热销榜中的游戏评论作为研究对象，根据建模需求筛选数据并进行人工标注，对数据集进行文本预处理，以适应模型需求。模型方面，使用 GitHub 开源平台中的 BERT 预训练模型，通过预训练模型+调参的方式，选择出最适合本文数据集的模型参数。第四部分为实验结果分析，以准确率、精确度、召回率和 F1 分数作为模型的评估指标进行评估。同时选择其他机器学习模型在相同数据集上进行测试，比较不同模型的性能。第五部分为总结与展望，总结在研究中所取得的成果和发现，以及选择模型的优缺点和特点等。讨论研究的不足之处和局限性，并探讨如何进一步改进该模型，从而提高模型的适应性和泛化能力，使其可以在产品评论、社交舆情分析中也发挥作用。本文的研究框架如图 1-2 所示：

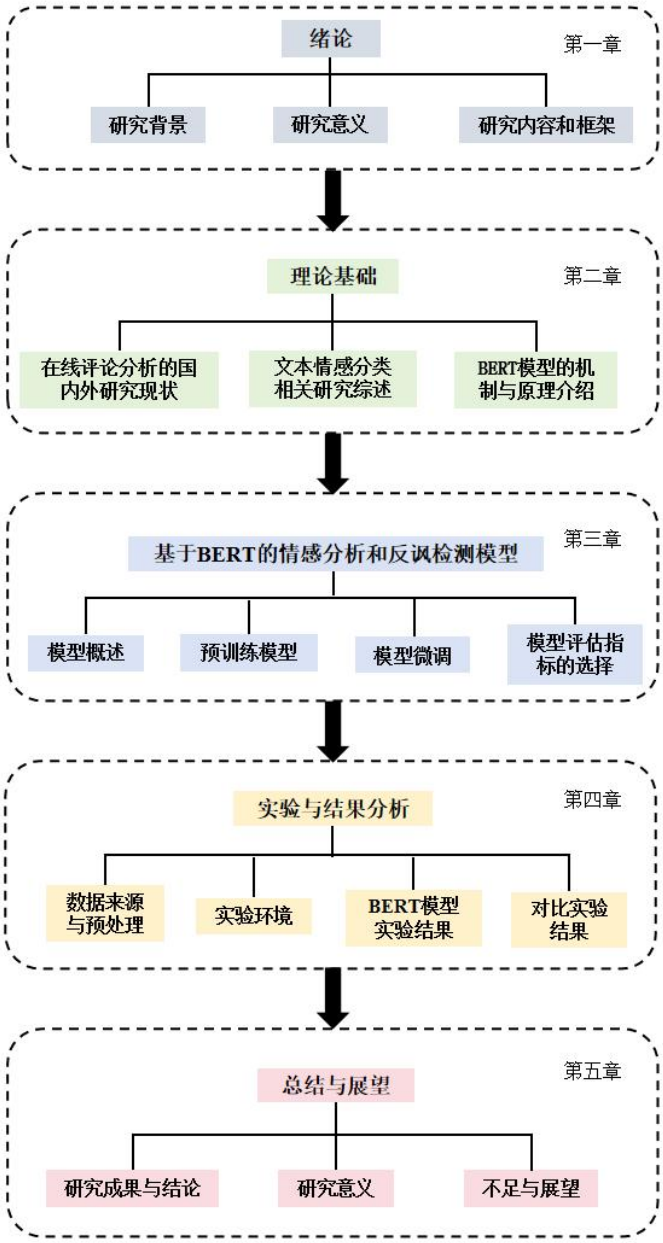
绪论
第一章
研究背景
研究意义
研究内容和框架
理论基础
第二章
在线评论分析的国内外研究现状
文本情感分类相关研究综述
BERT模型的机制与原理介绍
基于BERT的情感分析和反讽检测模型
第三章
模型概述
预训练模型
模型微调
模型评估指标的选择
实验与结果分析
第四章
数据来源与预处理
实验环境
BERT模型实验结果
对比实验结果
总结与展望
第五章
研究成果与结论
研究意义
不足与展望

图 1-2 研究框架图

2 理论基础

2.1 在线评论分析的国内外研究现状

在线评论指消费者在网站上对已购产品或服务发表的正面、中性或负面的评论，这些评论或表达个人情感，或客观描述产品属性，是消费者购买决策中最具影响力的信息来源之一[6]。对在线评论的分析和研究主要包含几个方面：

首先是情感分类领域的研究。文本情感分析又称意见挖掘，是指对带有情感色彩的主观性文本进行分析，挖掘其中蕴含的情感倾向，对情感态度进行划分[7]。为表达情感的强度，可以使用不同的测度，例如，常用的测量范围为[-1,1]，其中-1 表示最大负面情绪，1 表示最大正面情绪，0 表示中立态度[8][9]。Socher[10]等将情感评级分为 5 级，设定 0 为最大消极情绪，4 为最大积极情绪。王剑辉[11]等人提出了一种基于 LSTM+word2vec 的深度学习模型，将微博评论划分为正向和负向的评论后，进行了情感分析实验。而 Wu[12]等人则是提出了一种基于密集连接 LSTM 网络的系统，采用多任务学习策略甄别推特上的讽刺评论，还结合了几种类型的特征来提高模型性能。张财[13]等人提出了基于 word2vec 的支持向量机构建微博政务情感分析模型，以奥运会和中国足协相关评论为研究对象，为政务微博自动审核提供了新思路。Gangwar 和 Meh[14]等人使用了支持向量分类器（SVC）、决策树（DT）和朴素贝叶斯（NB）等机器学习算法，对政治推文进行了文本情感分析。

其次是主题识别。主题识别研究旨在从大规模评论数据中识别出关键主题或热点话题，并分析用户对这些主题的态度和观点。这有助于研究者了解用户的关注点、舆论倾向以及话题的发展演变过程，多出现于信息情报学领域。杨思洛[15]等人利用 BERTTopic 模型分析了我国信息资源管理领域的研究主题和演化趋势。李敏[16]等人以微博中后疫情时代的相关话题为研究对象，利用 TF-IDF 和 LDA 主题模型进行聚类分析，了解了后疫情时代背景下网民对疫情的关注焦点和情感态度，准确捕捉舆论动向和情感变化。周瑛[17]等人从京东商城爬取了部分产品评论，并对五种不同类别的产品评论进行了情感倾向分析，并研究消费者对产品属性的偏好和情感倾向间的关系。

主题检测是通过主题模型和情感分析技术识别评论中的主题和情感倾向，其中虚假信息识别和谣言监测以及反讽检测也是许多学者研究的领域。研究者通过分析评论的内容、来源、传播路径等信息，开发算法和模型来识别和应对虚假信息和谣言的传播。宋文哲[18]等人利用 BERT-BiLSTM-Att 的复合模型识别校园反诈信息，通过文本检测和情感分析等技术手段，提高校园反诈的准确性和效率。欧阳祺[19]等人于提出了一种基于 Bert-GNNs 异质图注意力网络的早期谣言检测算法，该模型可以充分利用文本全局-局部上下文语义关系和传播图的全局结构关系来增强特征表达。讽刺和隐喻识别也是情感分析中的一大挑战。讽刺作为互联网上常用的表现手法，使用轻蔑或调侃的语气表达出与所描述的事物相反的意思或效果，具有隐喻性、真实性的特点，往往难以识别[20]。王旭阳[21]等人针对中文微博评论进行了反讽识别，构建了一个包含反语、讽刺和非反讽的三分类数据集，并在 BERT 和双向长短期记忆网络模型上进行了测试。

2.2 文本情感分类相关研究综述

文本情感分析又称意见挖掘，是指对带有情感色彩的主观性文本进行分析，挖掘其中蕴含的情感倾向，对情感态度进行划分[6]。文本情感分析的一个基础划分就是对文本的情感进行极性分类，这种研究也被称为粗粒度情感分析。粗粒度情感又称情绪，通常包含正面情绪、负面情绪和中性情绪三类[22]。更高级的超出情感极性的分析里还会寻找更复杂的情感，如愤怒、厌恶、恐惧、快乐、悲伤和惊讶，是否含有讽刺意味等，这类分析被称为细粒度情感分析[23]。

目前在文本情感分析领域常用的方法主要有情感字典、机器学习以及深度学习方法。基于情感词典进行情感分析，首先需要对文本进行规范化处理，包括删除异常值和缺失值、去停用词和符号、转换大小写以及分词。情感词典可以是词语也可以是短语或句子，需要人工进行情感标注，并通过加权计算的方法得出文本的情感倾向。情感词典普遍运用简单且易于实现，但对于不同领域和不同的研究任务，需要收集大量的资料和情感倾向词语，因此人工成本过高。且因为互联网的快速发展，导致情感词典的滞后性比较强，不能及时满足文本分析的需求。为了进行大批量的文本处理和分析，机器学习应运而生。机器学习广泛地应用于自然语言处理和文本挖掘领域，研究人员可以利用大量有标记或无标记的文本进行监督学习、无监督学习或半监督学习。机器学习领域已经有很多比较成熟的模型用于文本情感分析，例如，逻辑回归、决策树、朴素贝叶斯、支持向量机（SVM）等。通常，研究者需要准备一个有标签的数据集，并选择合适的机器学习模型进行训练，最后通过调节参数选择最适合的模型特征进行情感分析。虽然机器学习模型比情感词典灵活，但还是依赖数据集的质量和特征工程。随着需求的不断增加，对文本情感分析的要求越来越高，普通的机器学习模型已经不能很好地处理一些复杂的数据，因此诞生了深度学习。深度学习也是机器学习的一个分支，是以人工神经网络为架构，对资料进行表征学习的算法。这种算法的好处是使用非监督学习或半监督学习的分层特征提取高效算法来代替手工特征提取。深度学习模型可以处理大规模的文本、图像和语音等数据，通过多层次的神经网络架构，提取出复杂的特征表示。深度学习模型还具有很强的适应性，可以适应不同的数据分布和任务需求。通过调整网络结构、超参数和训练方法，深度学习模型可以实现对不同数据和任务的灵活应用。

2.3 BERT 模型的机制与原理介绍

BERT（Bidirectional Encoder Representations from Transformers）模型[24]是谷歌公司在 2018 年引入的一种新的语言表示模型，这一模型旨在通过在所有层中联合地对左右上下文进行条件预训练，从未标记的文本中预训练深层双向表示。BERT 模型包括以下几个主要机制：双向 Transformer Encoder，多头注意力机制，词嵌入，掩码，以及 NSP。

2.3.1 双向 Transformer Encoder

Transformer 常用在自然语言处理领域，尤其是翻译领域。如图 2-1 所示：

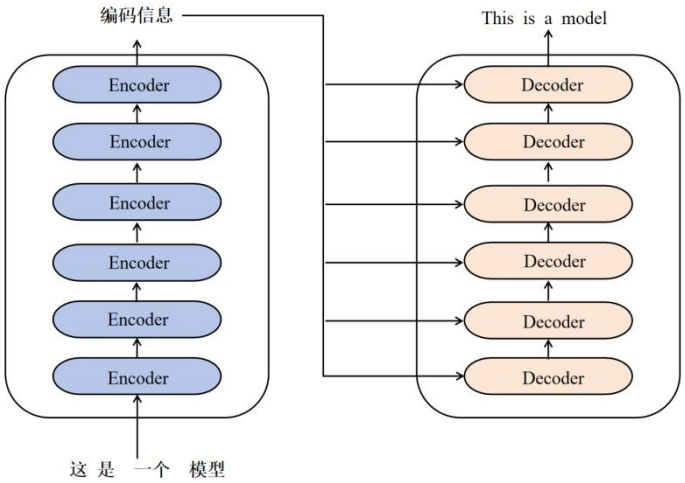

图 2-1 Transformer 编码器-解码器图解

BERT 采用双向 Transformer 编码器，在处理输入序列的同时可以获取序列的位置信息，在处理信息时更加全面地了解文本的语义和语境，从而提高模型的表征和泛化能力。

2.3.2 多头注意力机制

自注意力机制由 Vaswan[25]等人在 2017 年首次提出，它允许模型在处理一个序列时考

虑到序列中的元素与其他元素之间的关系。BERT 模型中所使用的是多头注意力机制（Multi-Head Attention）。如图 2-2 所示：左侧为 Encoder 模块，右侧为 Decoder 模块，而每个模块中都包含多头注意力机制。

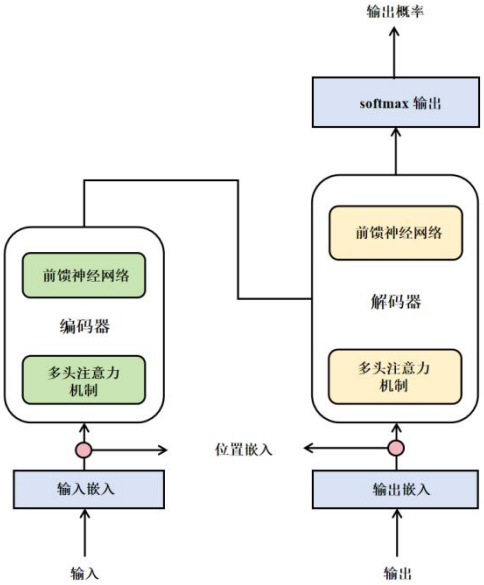

图 2-2 含有多头注意力机制的网络结构图

2.3.3 嵌入（Embedding）

词嵌入是指将文本中的每个单词表示为固定长度的分布式向量，相比于只包含统计信息的文本表征方法，词嵌入包含着语义信息，同一含义的词语在向量空间中是接近的[20]。BERT 的嵌入包括词嵌入（Token embedding）、片段嵌入（Segment Embedding）和位置嵌入（Position Embedding）。

2.3.4 掩码（Masked Language Model）

BERT 模型会在训练过程中随机遮盖 15%的 token，对这些 15%的词，有 80%的概率真的把它们替换成掩码，还有 10%的可能将它们替换成一个随机的词，还有 10%的概率保持不变，而是用它们来进行预测[24]。如图 2-3 所示：

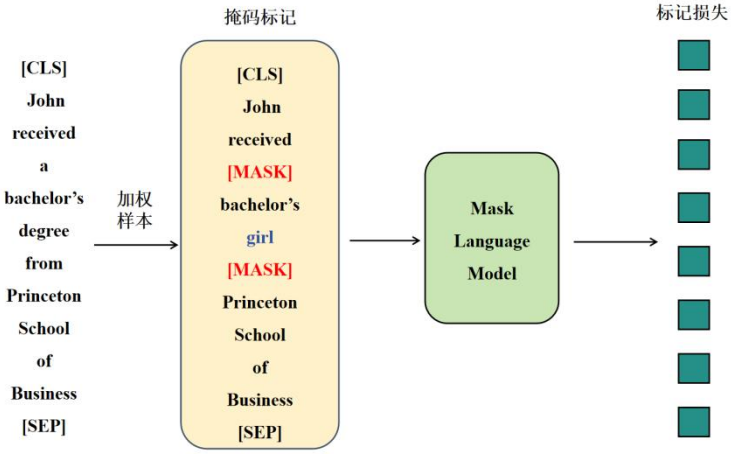

图 2-3 BERT 模型中的掩码图

2.3.5 NSP（Next Sentence Prediction）

NSP 可以使模型理解句子之间的关系，即预测两个句子是不是上下句的关系。首先，模型会从语料库中随机选择一对句子，这对句子若相邻则被标记为正样本 IsNext，不相邻被标记为负样本 NotNex，模型会根据输入序列中的上下文信息预测两个句子是否为上下句。

3 基于 BERT 的情感分析和反讽检测模型

3.1 模型概述

传统的研究通常聚焦于单一任务，例如情感分类或反讽检测。然而，本文提出了一种基于 BERT 的在线评论情感分类和反讽检测模型，同时兼顾了这两种任务，实现情感分析从粗粒度到细粒度的覆盖。通过 Python 爬虫采集需要的在线评论，并对数据进行清洗降噪和人工标注。对 BERT 预训练模型进行了微调，以适应特定的情感分类任务。将数据集分成训练集和测试集两部分，通过反向传播和优化算法不断更新参数，以最小化损失函数。剩下 20%作为测试集，通过评估指标评估模型在情感分类任务上的性能，模型的整体架构如图 3-1 所示。最后通过传统的机器学习方法和其他深度学习模型进行对比实验，验证 BERT 模型的有效性。

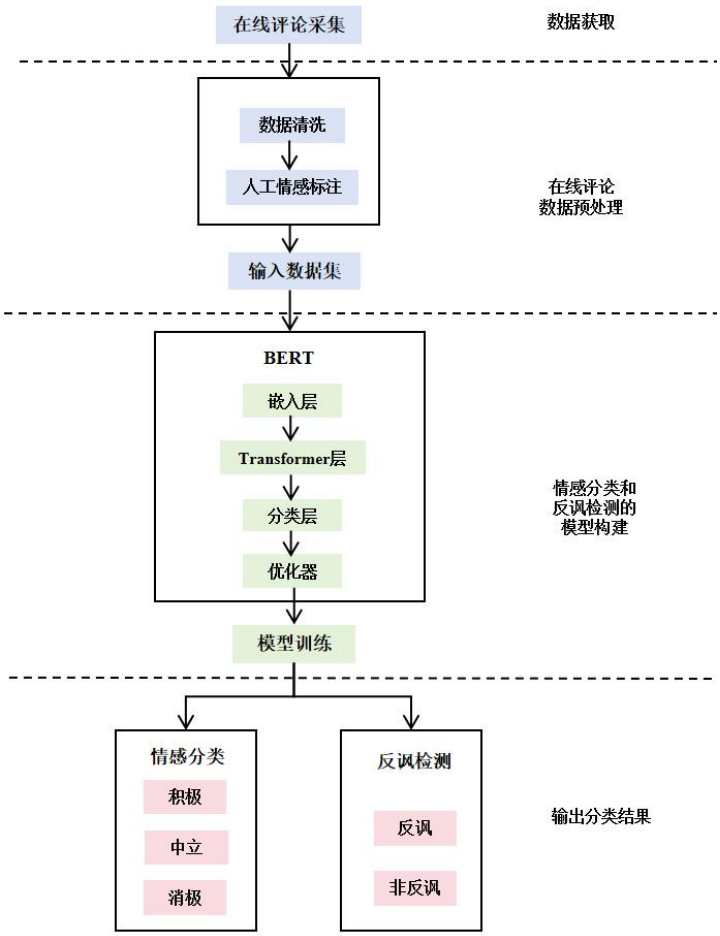

图 3-1 模型框架图

3.2 预训练模型

3.2.1 在线评论文本数据转换

模型首先对在线评论数据进行清洗和降噪后，将输入的评论文本转换为 BERT 模型可以理解的形式。在本文的任务中，将在线评论标记为积极评论 1、消极评论-1 或中立评论 0，用于情感三分类任务；再将每条在线评论标记为讽刺 1 或非讽刺评论 0，用于二元反讽检测任务。由于 BERT 模型只能识别特定的输入，因此需要将在线评论数据进行转换。例如，“Rubbish, poor optimisation for PC. Developers don't care about PC players. Don't waste your money.”这是一句评论文本，首先将文本分割成单词或子词，其中句子的开头用 CLS

标记，句末用 SEP 标记，这样是为了分隔不同的句子：['[CLS]','rubbish', ',', 'poor', 'optimization', 'for', 'pc', '.', 'developers', 'don', '"', 't', 'care', 'about', 'pc', 'players', '.', 'don', '"', 't', 'waste', 'your', 'money', '."[SEP]']，再将其转换为对应的词汇表中的索引：[17489, 1010, 3532, 23483, 2005, 7279, 1012, 8745, 2123, 1005, 1056, 2729, 2055, 7279, 2867, 1012, 2123, 1005, 1056, 5743, 2115, 2769, 1012]。其次，根据输入序列填充为固定长度，通常为 512 个 token，所以需要在原序列 23 个长度后在填充 489 个 0。因为在情感分类任务中，模型需要标签从 0 开始，因此原本的标签 1，-1 和 0 需要转换成 2，0 和 1。

模型在预训练的词汇表中找到特定单词的词嵌入向量，例如 rubbish 的词嵌入向量为[-0.23, 0.56, -0.11, ..., 0.04]。为了标记每个 token 在序列中的位置，需要添加位置编码。假设 rubbish 的位置为 0，则它的位置编码为[0.1, -0.2, 0.1, ..., 0.3]，将词的词向量和位置向量相加就成为了其最终向量。在输入模型前，还要对文本序列进行注意力掩码。注意力掩码是将填充的位置记为 0，实际标记位置记为 1。

3.2.2 编码和特征提取

在线评论甄别模型的核心机制是 Transformer 编码器，通过自注意力机制实现对在线评论序列的建模，自注意力机制的公式如 3-1 所示：

$$Attention(Q,K,V) = softmax(\frac{QK^T}{\sqrt{D_K}})V \tag{3-1}$$

以上文中的单词 rubbish 为例，查询向量 Q 表示为 rubbish 的词嵌入向量[0.1,−0.3,0.5,...,0.2]，键向量 K 和数值向量 V 也分别表示 rubbish 的词嵌入向量。接着对 Q 和 K 进行点积计算，并以维度 D 进行缩放，然后对结果进行 Softmax 操作。最后将 Softmax 后的注意力权重乘以 V，这就是 rubbish 这一词的最终表示。这些评论的 Attention 值序列会被传递到分类器层，通过全连接层和 Softmax 激活函数得到甄别的结果。全连接层的计算公式如 3-2 所示：

$$z = W \cdot h + b \tag{3-2}$$

在全连接层中，分类器的权重矩阵 W 将输入的特征向量映射到三维的向量空间，分别对应评论的积极、消极和中立情绪以及反讽和非反讽语义；h 为模型处理输入文本后的 CLS 标记的输出向量，b 用于调节全连接层的输出，z 表示未归一化的分类得分。在上述例子里，评论经过模型后得到 h=[−0.23,0.56,−0.11,...,0.04]，将这个向量输入全连接层，计算得到 z=[1.2,0.5,−0.3]。之后，Softmax 激活函数要计算评论情感的类别概率，其公式如 3-3 所示：

$$softmax(z_i) = \frac{e^{zi}}{\sum_{j=1}^{N} e^{zj}} \tag{3-3}$$

z_i 表示输出中的第 i 个得分，z_0 为消极情感评分，z_1 为中立情感得分，z_2 为积极情感得分。j 用来遍历所有情感类别，确保概率总和为 1。例如 z=[1.2,0.5,−0.3]在经过 Softmax 激活函数后得到的计算结果为[0.7,0.2,0.1]，表示模型认为该条评论有 70%的概率为消极情感，20%的概率为中立情感，10%的概率为积极情感，那么这条评论就会被预测为“消极”。

3.3 模型微调

BERT 是一个预训练的 Transformer 模型，在情感分类任务中通常需要进行微调以适应特定的任务，例如上文提到的情感标签的转，为了适应特定的数据集，需要调整适合的学习率。为了更好地平衡模型的收敛速度以及避免过拟合现象的出现，将使用较低的学习率。在遍历在线评论数据集的参数上，由于在线评论数据集的大小和计算机资源的限制，2-4 个 Epochs 足够取得好的效果。此外，为了提高模型的鲁棒性，本文使用 AdamW 优化器。AdamW 是 Adam 优化器的改良版，解决了在线评论数据集训练过程中权重参数过度增长

的问题，同时也可以防止过拟合。

3.4 模型评估指标的选择

本文选择以下四个评估指标评估模型，其中 TP（True Positive）表示将正面样本预测为正面，FP（False positive）表示将负面样本预测为正面，TN（True Negative）表示将负面样本预测为负面，而 FN（False Negative）表示将正面样本预测为负面。

准确度（Accuracy），指模型正确预测的样本数占总体样本数的比例，表示为式 3-4。准确率是一个直观的整体性能指标。然而，当类别分布不均衡时，准确率可能会产生误导性。

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \tag{3-4}$$

精确率（Precision），是指模型预测为正类的样本中实际为正类的比例，表示为式 3-5。在情感分类任务中，精确率高表示模型能较好地避免情感类别的误报，在反讽检测中亦然。

$$Precision = \frac{TP}{TP + FP} \tag{3-5}$$

召回率（Recall），是指是实际为正类的样本中被正确预测为正类的比例，表示为式 3-6。在情感分析任务中，召回率高表示模型能够识别出大部分的某一情感类别，减少漏报的情况。

$$Recall = \frac{TP}{TP + FN} \tag{3-6}$$

F1 分数是精确率和召回率的综合评估指标，表示为式 3-7。F1 分数在反讽监测和情感分类中非常重要，特别是在数据类别不平衡的情况下。

$$F1 - Score = 2 * \frac{Precision * Recall}{Precision + Recall} \tag{3-7}$$

4 实验与结果分析

4.1 数据来源与预处理

本文所使用的数据集来自 Steam 平台的游戏评论。Steam 平台作为作为全球最大的游戏平台之一，拥有庞大的用户群体和丰富的游戏资源，提供了丰富多样的游戏评论数据。其次，在游戏社区中，玩家对游戏内容的情感表达通常是直接且多样的。同时，游戏评论中经常出现网络热梗，也有很多玩家使用反讽或幽默的方式表达情感或批评游戏，因此作为研究对象时可以进行文本情感的粗细颗粒度分析。

Steam 商店的榜单概览实时更新平台上最畅销的和最热玩的游戏，本次研究主要选取热销榜前一百的游戏中具有代表性的游戏。这些游戏可以很大程度地覆盖不同类型游戏的评论，以便于研究分析。针对每款游戏，通过编写爬虫程序在 Steam 平台上获取了用户 ID、游戏时长、推荐与否、评论日期以及评论文本等信息，评论选择英文作为筛选的语言。

本次数据爬取使用的是 Python 中的 Selenium 库进行网页操作和数据提取，通过模拟用户在浏览器中的行为，包括打开网页、填写表单、点击按钮等操作来爬取需要的数据。为了不触发网站的反扒机制，在爬虫时还添加了 User-agent 和 Cookie 来模拟登录状态，同时设置了初始等待时间 30 秒，滚动页面时停留 5 秒。最终每款游戏收集 200-600 条评论，合计 5312 条数据，具体各类型游戏选择要素数据如表 4-1 所示。

表 4-1 各类型游戏选择要素数据表

	游戏名称	类型 1	类型 2	类型 3	类型 5	类型 6	类型 7	价格
1	EA SPORTS FC™ 24	足球	体育	动作	合作	阖家	本地多人	￥248
2	反恐精英	第一人称射击	多人	竞技	团队导向	电竞	战术	￥103
3	极限竞速：地平线 4	竞速	开放世界	驾驶	控制器	体育	欢乐	￥188
4	黎明杀机	恐怖	生存	血腥	心理恐怖	氛围	策略	￥93.75
5	熔炉密林	迷宫探索	动作冒险	角色扮演	中世纪	多人	在线合作	￥36
6	三国杀	纸牌	历史	3D	休闲	手绘	自走棋	免费
7	蟹蟹寻宝奇遇记	3D 平台	欢乐	氛围	卡通风格	可爱	剧情丰富	￥98
8	永劫无间	大逃杀	多人	武术	女性主角	生存	暴力	免费
9	Sid Meier's Civilization VI	策略	历史	回合制	战争	模拟	建造	￥220
10	Geometry Dash	困难	音乐	关卡编辑	节奏	街机	2D	￥18

首先对数据做描述性统计分析。根据分析结果，这一数据集中的最小值为 0，这可能是因为爬虫过程中由于网络或者停顿导致没有爬取到正确的数据。最长的评论长达 7994 个字符，评论长度平均为 756 个字符，但中位数却只有 169 个字符。根据图 4-1 可以明显看出，评论数据呈现右偏分布。

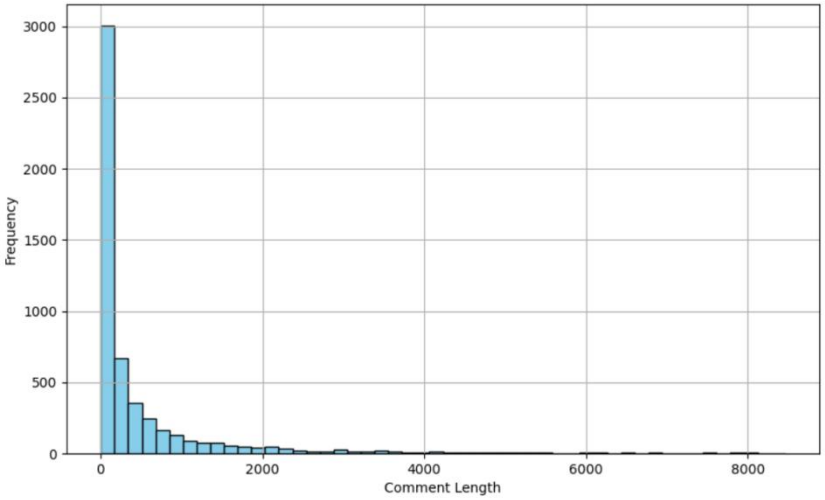

图 4-1 数据筛选前的评论字数分布图

因为文本很长的评论占少数，而且文本长度差异过大会导致模型准确度降低，因此本次研究剔除字符长度大于 300 个字符的评论。其次，因为游戏时长较短的用户可能不具有代表性，为了提高数据分析的质量和可信度，只保留游戏时长大于 100 小时的评论。图 4-2 为筛选后的评论长度和频率。经过系统的数据处理，用于数据预处理的评论共 1323 条。

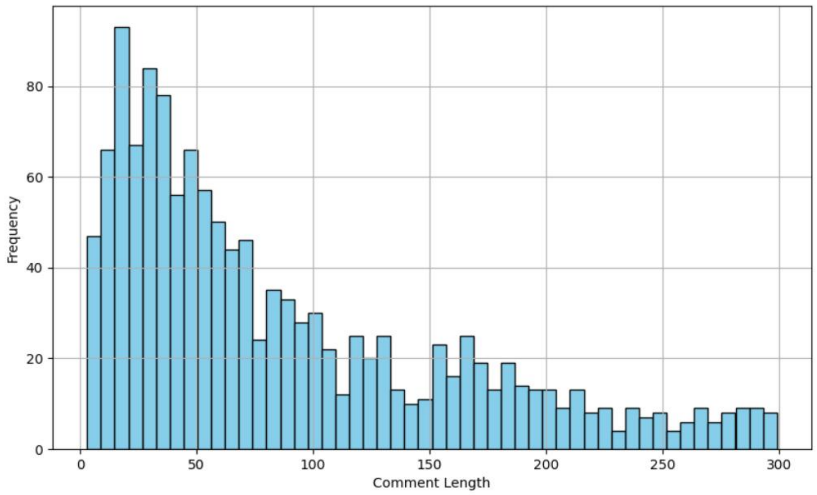

图 4-2 评论筛选后的评论字数分布图

根据本文的任务，为数据集进行人工标注。该数据集共两组标签，分别是情感标签和反讽标签。其中情感标签有三类分别是 1 表示积极评论，0 表示中立评论，-1 表示消极评论。在反讽标签中，1 表示文本含有反讽语义，0 表示文本不含反讽语义。表 4-2 展示了部分数据集：

表 4-2 有人工标注的部分数据集

评论	情感标签	反讽标签
Rubbish, poor optimisation for PC. Developers don't care about PC players. Don't waste your money.	-1	0
Good game like always, especially the gambling mechanics designed for kids are something that is really innovative!	-1	1

续表 4-2 有人工标注的部分数据集

评论	情感标签	反讽标签
Purchased Ultimate Edition. Recieved none of the things from the Ultimate edition. EA Launcher says i purchased the standard edition. I despise this company	-1	0
DO NOT BUY FIFA POINTS DO NOT BUY FIFA POINTS DO NOT BUY FIFA POINTS DO NOT BUY FIFA POINTS DO NOT BUY FIFA POINTS	-1	0
Affects Mental Health	-1	0
If you value your mental stability, please stray far away from not only this game but EA Sports vancoover as whole. These stingy frickers dont give two whoopties about their game and only want you for your wallet.	-1	0

根据评论标签对评论进行统计和分类，可以发现数据集中共有 482 条积极评论，458 条中立评论，318 条消极评论，评论情感分布趋于平衡。然而，数据集中仅有 65 条评论被标记为含有反讽语义，剩下 1258 条评论则不含有反讽语义，不平衡的数据集可能会对后期建模产生影响。表 4-3 展示了数据集的标签矩阵：

表 4-3 数据集的标签矩阵表

数据集	积极评论	中立评论	消极评论	合计
含讽刺的评论	2	63	0	65
不含讽刺的评论	482	458	318	1258
合计	484	521	318	1323

图 4-3 所展示的是该数据集的词云图，可以看到作为游戏评论，game 这一词在评论中处于最重要的位置；而与游戏相关的词 play，fun，hour，time，等次出现的频率也很高；good，hate 等明显的情感词也处于比较重要的地位。

图 4-3 游戏评论数据集词云图

4.2 实验环境

本文使用的实验平台是 Anaconda 环境，硬件为 WDCPCSN730SDBPNTY-512G-1101，内存为 16G，处理器为 AMD Ryzen 5 4600U。编码采用 Python3.7 版本，模型所使用的库为 torch1.13.1+cpu，transformers4.30.2，pandas1.3.5 和 scikit-learn1.0.2。两个模型的参数如表 4-4 所示：

表 4-4 BERT 模型的参数设置

名称	释义	情感分类参数	反讽检测参数
max_len	输入文本序列的最大长度	512	512
batch_size	每个训练批次中包含的样本数量	8	8
lr（learning rate）	优化器的学习率	2e-5	2e-5
num_epochs	训练模型时便利整个数据集的次数	4	4
num_labels	指定分类任务的类别数量	3	2
AdamW	优化器	默认值	默认值
DataLoader	将数据集组织成小批量的数据	默认值	默认值
model_name	预训练 BERT 模型的名称	bert-base-uncased	bert-base-uncased

对于情感分类和反讽检测任务，我们都将输入文本序列的最大长度设置为 512，超过这个长度的文本将被截断或者进行填充。两个模型都选择了相同的 batch_size 和相同的学习率，即每个训练批次包含 8 个样本和 2e-5 的学习率，选择了训练模型便利数据集 4 次。在情感分类任务中，我们将类别数量设置为 3，而在反讽检测任务中，我们将类别数量设置为 2。

4.3 BERT 模型实验结果

模型的情感分类结果如表 4-5 所示。总体来说，模型在分类积极评论和消极评论时表现良好，几乎所有被分类的评论都是真正的积极或消极评论。然而，对于中立评论的分类，模型的性能较低，只有一半的中立评论被正确分类。模型在整体上具有一定的性能，但对于中立评论的识别效果有待提高。

表 4-5 情感分类模型的评估结果表

	Accuracy	Precision	Recall	F1 Score
Class 1	0.82	1	0.82	0.90
Class 0	0.54	1	0.54	0.70
Class -1	0.75	1	0.75	0.86
Overall	0.73	0.70	0.70	0.70

模型的反讽监测结果如表 4-6 所示。在这个评估结果中，大约 95%的样本被正确分类，被分类为正类别的样本中，约有 93.4%确实是正类别，表明分类器在正类别的预测中具有相对较高的准确性；召回率为 95%，这说明分类器对于正类别的识别能力也是相对较高的；F1 分数为 93%，这表明分类器在整体上具有较好的平衡性能，能够在准确率和召回率之间取得良好的折中。

表 4-6 反讽监测模型的评估结果表

	Accuracy	Precision	Recall	F1 Score
Overall	0.95	0.93	0.95	0.93

4.4 对比实验结果

为了进一步验证模型的效果，我们选择了其他机器学习模型在相同数据集上进行了验证。在情感分类方面，我们选择了逻辑回归模型（LR）、支持向量机（SVM）和长短期记忆模型（LSTM）做对比实验，并输出评估结果，结果如表 4-7 所示。

表 4-7 情感分类任务模型对比结果

模型名	Accuracy	Precision	Recall	F1-score
LR	0.63	0.61	0.59	0.59
SVM	0.62	0.59	0.58	0.58
LSTM	0.55	0.55	0.54	0.53
BERT	0.73	0.7	0.7	0.7

在图 4-4 中，我们可以更加直观地看到不同模型在情感分类任务上的表现能力。在本文的数据集中，BERT 模型在四个指标中都表现的最好，其次是逻辑回归模型和支持向量机，长短期记忆模型在改数据集上表现最差。这是因为 BERT 模型作为预训练模型，有大量的数据集训练作为基础，相对于传统的机器学习模型和简单的神经网络模型具有更复杂的结构和更多的参数，能够更好地拟合复杂的文本数据。

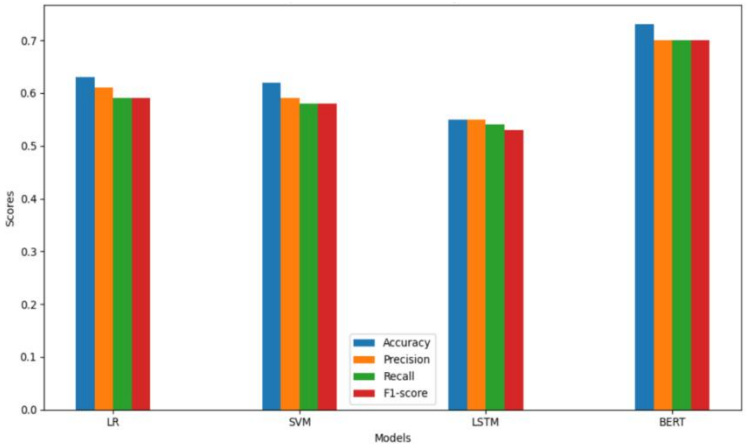

图 4-4 情感分类模型性能比较图

对于反讽检测任务，我们选择了朴素贝叶斯（NB）、决策树（DT）、多层感知器（MLP）等模型做对比实验。在反讽监测任务中，标记为含有反讽的评论仅占总体评论的 5%左右，数据集呈现不平衡状态。由于其简单的模型结构和假设，朴素贝叶斯对于较小的训练集和不平衡数据集也能够给出合理的分类结果。决策树通过递归将数据集划分为不同的子集，并根据特征值进行分类，在处理不平衡数据集时也是一个合适的选择。MLP 由多个神经元组成，具有较强的拟合能力，能够适应各种复杂的非线性关系。模型的评估结果如表 4-8 所示。

表 4-8 反讽检测任务模型对比结果

模型名	Accuracy	Precision	Recall	F1-score

续表 4-8 反讽检测任务模型对比结果

模型名	Accuracy	Precision	Recall	F1-score
NB	0.88	0.25	0.53	0.34
DT	0.93	0.2	0.1	0.2
MLP	0.94	0.5	0.2	0.29
BERT	0.95	0.93	0.95	0.93

在图 4-5 中，我们可以更加直观地看到不同模型在反讽检测任务上的表现能力。显然，BERT 模型在各指标上的表现都很出色。MLP 模型的准确率稍高于朴素贝叶斯和决策树，但精确率、召回率和 F1 分数略低。朴素贝叶斯和决策树模型的性能较为有限，其中朴素贝叶斯的召回率相对较高，但精确率较低，决策树的精确率和召回率均较低。

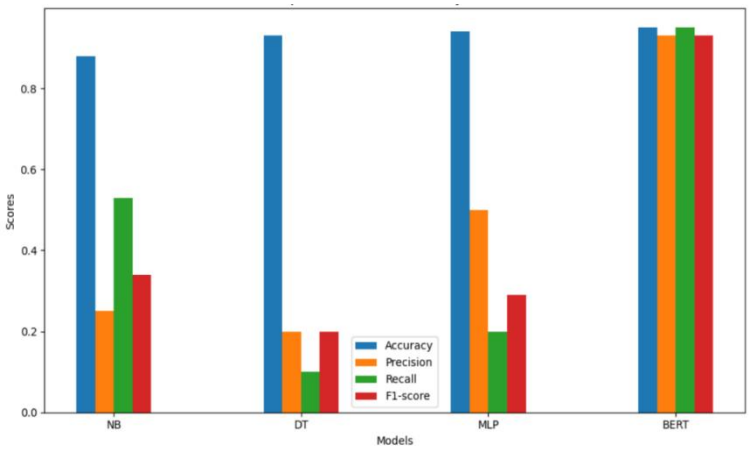

图 4-5 反讽检测模型性能比较图

5 总结与展望

5.1 研究成果与结论

本文提出了一种基于 BERT 预训练模型的文本分类模型，以 Steam 平台为例，对热销榜上的游戏评论进行了情感分析和反讽检测。根据模型评估结果可以看出，在相同的数据集上，模型对游戏评论的反讽检测性能显然优于情感分类，这一发现在营销领域尤为重要，因为评论效价对于产品的市场反馈和品牌形象至关重要。本文所使用的数据及标签为人工标注，而数据集的标注质量可能直接影响模型训练结果。根据模型的结果可知，在反讽检测任务中，数据集的标注质量较高，模型能够准确识别反讽语句。

5.2 研究意义

在线评论的情感分析对商家和消费者都有重要意义。对消费者而言，在线评论的情感分类可以帮助他们快速获取其他用户的真实评价，减少选择和购买决策的时间成本。消费者可以更准确地判断产品的实际性能，从而做出更加明智的购买决策。

准确识别在线评论中的真实情感和反讽语义，还可以帮助企业更好地了解市场需求，迅速识别产品的优势和不足，进而优化产品设计和改进服务质量。这种实时反馈机制可以提升用户满意度和品牌忠诚度，从而提高产品竞争力。在营销活动中，企业可以精准定位目标受众，制定更有效的营销策略。例如，通过分析评论中的情感信息，企业可以识别出不同用户对特定产品或服务的情感共鸣，从而开展针对性的市场推广活动。情感分析还可以帮助企业监测品牌声誉，及时应对负面评价，维护良好的品牌形象

5.3 不足与展望

数据集的不平衡可能是导致这一现象的原因之一。在本文中，数据集在情感分类上趋于平衡，在反讽分类上呈现不平衡趋势，然而模型输出结果却相反。这很有可能是因为 BERT 模型具有强大的泛化能力，可以有效处理不平衡的数据集。虽然数据集的情感标签更平衡，但本文使用的 BERT 预训练模型为情感二分类模型，而本文的使用的数据集为情感三分类任务，因此可能存在一些更加复杂的模式和语义需要模型更深层次的理解。

在未来的研究中，可以对模型的性能进行进一步的改进和研究。例如针对情感分类任务进行更细粒度的标注，从而探索出更适合游戏评论的情感分类模型，如 RoBERta 模型和 ALBERT 模型等。同时，由于缺少开源的游戏评论数据集，在未来的研究中可以重点关注这一缺口。在更多优质的数据集上改进标注的质量，并利用主动学习、迁移学习、半监督学习等技术提高模型在游戏评论甄别上的性能，从而更好地服务于游戏营销和品牌建设。此外，还可以将模型应用到其他社交平台，验证不同场景下模型的表现，如其他商品评价、电影评论等。此外，还应该进一步针对不同主题的专业术语、网络用语、比喻和隐喻等进行深入研究，结合语义分析和自然语言处理的最新进展，提高模型对复杂语言现象的处理结果。

致　谢

首先，我要感谢郑州大学为我提供了优越的学习环境和丰富的资源，使我能够顺利完成这篇研究工作。在这里，我得到了宝贵的知识和技能，为我的学术和职业生涯奠定了坚实的基础。

其次，我要特别感谢我的导师丁老师。在整个研究过程中，您的悉心指导、无私支持和耐心帮助使我能够克服重重困难，完成这项课题。您的专业知识和学术精神深深地影响了我，让我受益匪浅。

最后，我要深深感谢我的父母。你们的理解、鼓励和支持是我前进的最大动力。无论是生活上的关怀还是精神上的鼓舞，你们始终在我身边，为我的每一步成长提供了无尽的支持和力量。

衷心感谢所有在我学习和研究过程中给予我帮助和支持的人，感谢你们的付出和关爱！

参考文献

[1] 谭荧,张进,夏立新.社交媒体情境下的情感分析研究综述[J].数据分析与知识发现,2020,4(01):1-11.

[2] Scheufele D A, Krause N M. Science audiences, misinformation, and fake news[J]. Proceedings of the National Academy of Sciences, 2019, 116(16): 7662-7669.

[3] Sivandzadeh G R, Askari H, Safarpour A R, et al. COVID-19 infection and liver injury: Clinical features, biomarkers, potential mechanisms, treatment, and management challenges[J]. World Journal of Clinical Cases, 2021, 9(22): 6178-6190.

[4] 杜洪涛,孟庆国,王君泽.互联网数据在舆情研判中的效用研究[J].中国软科学,2016,(04):34-44.

[5] 赵爽, 马君. 算法管理与员工创造力：借算法之势而创还是困数字牢笼而息[J/OL]. 系统管理学报, 2024, 1-26.

[6] 武瑞娟,刘京京,张志帅.幽默负面评论、商家回复与消费者对商家的态度[J].营销科学学报,2023,3(03):102-120.

[7] 韩坤, 潘宏鹏, 刘忠轶. 融合 BERT 多层次特征的短视频网络舆情情感分析研究[J]. Journal of Frontiers of Computer Science & Technology, 2024, 18(4).

[8] Li S T, Tsai F C. A Fuzzy Conceptualization Model for Text Mining with Application in Opinion Polarity Classification[J].Knowledge-Based Systems, 2013, 39: 23-33.

[9] Martín-Valdivia M T, Martínez-Cámara E, Perea-Ortega J M, et al. Sentiment Polarity Detection in Spanish Reviews Combining Supervised and Unsupervised Approaches[J]. Expert Systems with Applications, 2013, 40(10): 3934-3942.

[10] Socher R, Perelygin A, Wu J Y, et al. Recursive Deep Models for Semantic Compositionality Over a Sentiment Treebank[C]//Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing. 2013: 1631-1642.

[11] 王剑辉, 闫芳序. 基于 LSTM+Word2vec 的微博评论情感分析[J/OL]. 沈阳师范大学学报(自然科学版), 2024, (02): 138-144.

[12] Wu C, Wu F, Wu S, et al. Thu_ngn at semeval-2018 task 3: Tweet irony detection with densely connected LSTM and multi-task learning[C]//Proceedings of The 12th International Workshop on Semantic Evaluation. 2018: 51-56.

[13] 张财, 马自强, 闫博. 基于机器学习的政务微博情感分析模型设计[J/OL]. 计算机工程, 2024, 1-14.

[14] Gangwar A, Mehta T. Sentiment analysis of political tweets for Israel using machine learning[C]//Proceedings of the International Conference on Machine Learning and Big Data Analytics. Cham: Springer International Publishing, 2022: 191-201.

[15] 杨思洛, 于永浩. 基于 BERTopic 模型的国内信息资源管理研究主题挖掘与演化分析[J/OL]. 情报科学, 2024, 1-18.

[16] 李敏, 项朝辉. 后疫情时代网络舆情情感分析和主题识别[J]. 电脑知识与技术, 2024, 20(02): 9-12.

[17] 周瑛, 张晓宇, 虞小芳. 基于产品评论挖掘的消费者偏好分析[J]. 情报科学, 2022, 40(01): 58-65.

[18] 宋文哲,陈宇琪,强宇韬,等.基于复合型 BERT 模型的文本检测及情感分析的校园反诈大数据综合平台开发[J].网络安全技术与应用,2024,(05):61-65.

[19] 欧阳祺,陈鸿昶,刘树新,等.基于 Bert-GNNs 异质图注意力网络的早期谣言检测[J].电子学报,2024,52(01):311-323.

[20]刘其龙, 李弼程, 黄志勇. CCSD:面向话题的讽刺识别方法[J/OL]. 计算机科学, 2024, 1-10.
[21]王旭阳, 戚楠, 魏申酉. 融合 BERT 和双向长短时记忆网络的中文反讽识别研究[J/OL]. 计算机工程与应用, 2024, 1-8.
[22]BIRJALI M, KASRI M, BENI-HSSANE A. A comprehensive survey on sentiment a nalysis approaches, challenges and trends [J]. Knowl Based Syst, 2021, 226: 107134
[23]EKMAN P, FRIESEN W V. Constants across cultures in the face and emotion [J].JP ersSocPsychol, 1971, 17(2): 124-129.
[24]Devlin J, Chang M W, Lee K, et al. Bert: Pre-training of deep bidirectional transfor mers for language understanding[J]. arxiv preprint arxiv:1810.04805, 2018.
[25]VASWANI A, SHAZEER N, PARMAR N, et al. Attention is all you need[J]. Advan ces in neural information processing systems, 2017, 30.

附　录

游戏评论爬虫代码

```
from selenium import webdriver
from selenium.webdriver.chrome.options import Options
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
import pandas as pd
import time
chrome_options = Options()
chrome_options.add_argument("--disable-gpu")
chrome_options.add_argument("--no-sandbox")
chrome_options.add_argument("--disable-dev-shm-usage")

user_agent = "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,
like Gecko) Chrome/123.0.0.0 Safari/537.36"
chrome_options.add_argument(f"user-agent={user_agent}")
chromedriver_path = "D:\python\chromedriver-win64
(2)\chromedriver-win64\chromedriver.exe"
driver = webdriver.Chrome(options=chrome_options)
url=" "
driver.get(url)
initial_wait_time = 30
WebDriverWait(driver,
initial_wait_time).until(EC.presence_of_element_located((By.CLASS_NAME,
"apphub_CardTextContent")))
cookies = [
    {'name': 'recently VisitedAppHubs', 'value': '1172470'},
    {'name': 'timezoneOffset', 'value': '28800,0'},
    {'name': 'browserid', 'value': '3243226958252487081'},
    {'name': 'strInventoryLastContext', 'value': '753_6'},
    {'name': 'steamLoginSecure', 'value':
'76561199252225915%7C%7CeyAidHlwIjogIkpXVCIsICJhbGciOiAiRWREU0EiIH0.eyAiaX
NzIjogInI6MTdCNl8yNDFCRDRFRV9DMjFCNyIsICJzdWIiOiAiNzY1NjExOTkyNTIyMjU5
MTUiLCAiYXVkIjogWyAid2ViOmNvbW11bml0eSIgXSwgImV4cCI6IDE3MTE0MjkzMDA
sICJuYmYiOiAxNzAyNzAyNzI0LCAiaWF0IjogMTcxMTM0MjcyNCwgImp0aSI6ICIwRUNC
XzI0Mjc4QTQ3X0Q0QUI4IiwgIm9hdCI6IDE3MTAzOTAxNTUsICJydF9leHAiOiAxNzI4Nzc
yOTQ1LCAicGVyIjogMCwgImlwX3N1YmplY3QiOiAiNjEuNzUuMTcwLjUiLCAiaXBfY29u
ZmlybWVyIjogIjYxLjc1LjE3MC41IiB9.RSB5kyXeVpPNmd-p7HnIo3YYQVTcyOKBvwJkDT
2i3HqMWAFX94zlz8_tMugi644VR8P8PFpFco_aVImjllIWBA'},
    {'name': 'sessionid', 'value': '9b215c9598ef5b1e8871fa37'},
    {'name': 'app_impressions', 'value':
'1172470@2_9_100010_|1172470@2_9_100010_|1172470@2_9_100010_|1172470@2_9_100
010_|1172470@2_9_100010_|1172470@2_9_100010_|1172470@2_9_100010_|1172470@2_9
```

```
_100010_|1172470@2_9_100010_|1172470@2_9_100010_|1172470@2_9_100010_|1172470
@2_9_100010_|1172470@2_9_100010_|1172470@2_9_100010_|1172470@2_9_100010_'}
]
for cookie in cookies:
    driver.add_cookie(cookie)

initial_wait_time = 30
WebDriverWait(driver,
initial_wait_time).until(EC.presence_of_element_located((By.CLASS_NAME,
"apphub_CardTextContent")))
scroll_pause_time = 5
last_height = driver.execute_script("return document.body.scrollHeight")
while True:
    driver.execute_script("window.scrollTo(0, document.body.scrollHeight);")
    time.sleep(scroll_pause_time)
    new_height = driver.execute_script("return document.body.scrollHeight")
    if new_height == last_height:
        break
    last_height = new_height

reviews = driver.find_elements(By.CLASS_NAME, "apphub_Card")
reviews_list = []
for review in reviews:
    try:
        user_id = review.find_element(By.CLASS_NAME,
"apphub_CardContentAuthorName").text
    except:
        user_id = "N/A"
    try:
        comment_text = review.find_element(By.CLASS_NAME,
"apphub_CardTextContent").text
    except:
        comment_text = "N/A"
    try:
        playtime = review.find_element(By.CSS_SELECTOR, ".hours").text
    except:
        playtime = "N/A"
    try:
        recommendation = review.find_element(By.CLASS_NAME, "title").text
    except:
        recommendation = "N/A"

    reviews_list.append({"用户 ID": user_id, "评论文本": comment_text, "游戏时长":
playtime, "推荐与否": recommendation})
```

```
file_path = (" ")
df = pd.DataFrame(reviews_list)
df.to_csv(file_path, index=False, encoding='utf-8-sig')
print("评论已保存到 CSV 文件:", file_path)
driver.quit()
```

文本预处理代码

```
import pandas as pd
from sklearn.model_selection import train_test_split
from transformers import BertTokenizer
import torch
import re
import nltk
from nltk.corpus import stopwords

nltk.download('stopwords')
stop_words = set(stopwords.words('english'))

def preprocess_text(text):
    text = text.lower()
    text = re.sub(r"[^a-zA-Z0-9\s]", "", text)
    text = ' '.join(word for word in text.split() if word not in stop_words)
    return text

data_path = " "
df = pd.read_csv(data_path)
df = df.dropna()
df['评论'] = df['评论'].astype(str)
df['评论'] = df['评论'].apply(preprocess_text)

train_texts, test_texts = train_test_split(df['评论'].tolist(), test_size=0.2, random_state=42)
model_name = "bert-base-uncased"
tokenizer = BertTokenizer.from_pretrained(model_name)

max_len = 512
train_encodings = tokenizer(train_texts, truncation=True, padding=True, max_length=max_len)
test_encodings = tokenizer(test_texts, truncation=True, padding=True, max_length=max_len)
train_inputs = torch.tensor(train_encodings.input_ids)
train_masks = torch.tensor(train_encodings.attention_mask)
test_inputs = torch.tensor(test_encodings.input_ids)
test_masks = torch.tensor(test_encodings.attention_mask)
processed_train_df = pd.DataFrame({
    'input_ids': train_inputs.tolist(),
    'attention_mask': train_masks.tolist()
})
```

```
processed_train_df.to_csv("processed_train_data.csv", index=False)

processed_test_df = pd.DataFrame({
    'input_ids': test_inputs.tolist(),
    'attention_mask': test_masks.tolist()
})
processed_test_df.to_csv(" ", index=False)
```

用于情感分类的 BERT 模型代码

```
import pandas as pd
from sklearn.model_selection import train_test_split
from torch.utils.data import DataLoader, TensorDataset
from transformers import BertTokenizer, BertForSequenceClassification, AdamW
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import torch

data_path = " "
df = pd.read_csv(data_path)
texts = df['评论'].tolist()
sentiments = df['情感'].tolist()

model_name = "bert-base-uncased"
tokenizer = BertTokenizer.from_pretrained(model_name)
model = BertForSequenceClassification.from_pretrained(model_name, num_labels=3)
sentiments = [sentiment + 1 for sentiment in sentiments]

train_texts, test_texts, train_sentiments, test_sentiments = train_test_split(texts, sentiments,
test_size=0.2, random_state=42)
max_len = 512
train_encodings = tokenizer(train_texts, truncation=True, padding=True, max_length=max_len)
test_encodings = tokenizer(test_texts, truncation=True, padding=True, max_length=max_len)

train_inputs = torch.tensor(train_encodings.input_ids)
train_masks = torch.tensor(train_encodings.attention_mask)
train_sentiments = torch.tensor(train_sentiments)
test_inputs = torch.tensor(test_encodings.input_ids)
test_masks = torch.tensor(test_encodings.attention_mask)
test_sentiments = torch.tensor(test_sentiments)
train_dataset = TensorDataset(train_inputs, train_masks, train_sentiments)
train_loader = DataLoader(train_dataset, batch_size=8, shuffle=True)
test_dataset = TensorDataset(test_inputs, test_masks, test_sentiments)
test_loader = DataLoader(test_dataset, batch_size=8)

optimizer = AdamW(model.parameters(), lr=5e-5)
num_epochs = 4
```

```
for epoch in range(num_epochs):
    model.train()
    for batch in train_loader:
        optimizer.zero_grad()
        input_ids, attention_mask, labels = batch
        outputs = model(input_ids, attention_mask=attention_mask, labels=labels)
loss = outputs.loss
        loss.backward()
        optimizer.step()

model.eval()
predicted_labels = []
true_labels = []
with torch.no_grad():
    for batch in test_loader:
        input_ids, attention_mask, labels = batch
        outputs = model(input_ids, attention_mask=attention_mask)
        _, predicted = torch.max(outputs.logits, 1)
        predicted_labels.extend(predicted.tolist())
        true_labels.extend(labels.tolist())

accuracy = accuracy_score(true_labels, predicted_labels)
precision = precision_score(true_labels, predicted_labels, average='macro')
recall = recall_score(true_labels, predicted_labels, average='macro')
f1 = f1_score(true_labels, predicted_labels, average='macro')

print("Class-wise Evaluation:")
for label in range(3):
    label_indices = [i for i, l in enumerate(true_labels) if l == label]
    label_true = [true_labels[i] for i in label_indices]
    label_pred = [predicted_labels[i] for i in label_indices]
    label_accuracy = accuracy_score(label_true, label_pred)
    label_precision = precision_score(label_true, label_pred, average=None)
    label_recall = recall_score(label_true, label_pred, average=None)
    label_f1 = f1_score(label_true, label_pred, average=None)
        print(f"Class {label}:")
    print(f"Accuracy: {label_accuracy}")
    print(f"Precision: {label_precision}")
    print(f"Recall: {label_recall}")
    print(f"F1 Score: {label_f1}")
    print()

print("Overall Evaluation:")
print(f'Accuracy: {accuracy}')
print(f'Precision: {precision}')
```

```
print(f'Recall: {recall}')
print(f'F1 Score: {f1}')
```

用于反讽检测的 BERT 模型代码

```
import pandas as pd
from sklearn.model_selection import train_test_split
from torch.utils.data import DataLoader, TensorDataset
from transformers import BertTokenizer, BertForSequenceClassification, AdamW
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import torch

data_path = " "
df = pd.read_csv(data_path)
texts = df['评论'].tolist()
sentiments = df['反讽'].tolist()

model_name = "bert-base-uncased"
tokenizer = BertTokenizer.from_pretrained(model_name)
model = BertForSequenceClassification.from_pretrained(model_name, num_labels=2)
train_texts, test_texts, train_sentiments, test_sentiments = train_test_split(texts, sentiments,
test_size=0.2, random_state=42)

max_len = 512
train_encodings = tokenizer(train_texts, truncation=True, padding=True, max_length=max_len)
test_encodings = tokenizer(test_texts, truncation=True, padding=True, max_length=max_len)
train_inputs = torch.tensor(train_encodings.input_ids)
train_masks = torch.tensor(train_encodings.attention_mask)
train_sentiments = torch.tensor(train_sentiments)
test_inputs = torch.tensor(test_encodings.input_ids)
test_masks = torch.tensor(test_encodings.attention_mask)
test_sentiments = torch.tensor(test_sentiments
train_dataset = TensorDataset(train_inputs, train_masks, train_sentiments)
train_loader = DataLoader(train_dataset, batch_size=8, shuffle=True)
test_dataset = TensorDataset(test_inputs, test_masks, test_sentiments)
test_loader = DataLoader(test_dataset, batch_size=8)


optimizer = AdamW(model.parameters(), lr=2e-5)
num_epochs = 4
for epoch in range(num_epochs):
    model.train()
    for batch in train_loader:
        optimizer.zero_grad()
        input_ids, attention_mask, labels = batch
        outputs = model(input_ids, attention_mask=attention_mask, labels=labels)
```

```
        loss = outputs.loss
        loss.backward()
        optimizer.step()

model.eval()
predicted_labels = []
true_labels = []
with torch.no_grad():
    for batch in test_loader:
        input_ids, attention_mask, labels = batch
        outputs = model(input_ids, attention_mask=attention_mask)
        _, predicted = torch.max(outputs.logits, 1)
        predicted_labels.extend(predicted.tolist())
        true_labels.extend(labels.tolist())

overall_accuracy = accuracy_score(true_labels, predicted_labels)
overall_precision = precision_score(true_labels, predicted_labels, average='weighted')
overall_recall = recall_score(true_labels, predicted_labels, average='weighted')
overall_f1 = f1_score(true_labels, predicted_labels, average='weighted')

print("\nOverall Evaluation:")
print(f'Overall Accuracy: {overall_accuracy}')
print(f'Overall Precision: {overall_precision}')
print(f'Overall Recall: {overall_recall}')
print(f'Overall F1 Score: {overall_f1}')
```